

TITLE :Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Conclusion

This Java program successfully demonstrates the implementation of abstraction through abstract classes and abstract methods. By defining base classes like Payment and FoodOrder, essential properties and concrete methods are shared while forcing subclasses to provide custom implementations for abstract methods such as processPayment() and calculateBill(). This design prevents direct instantiation of incomplete base classes, enforces structural contracts across subclasses, and promotes compile-time abstraction, resulting in a flexible, modular, and maintainable codebase.

Exercise

1. Create an abstract **Payment class** and implement Credit Card and UPI payment classes.?

Code:

PTO

```

1  abstract class Payment {
2      protected double amount;
3      public Payment(double amount) {
4          this.amount = amount;
5      }
6      abstract void processPayment();
7      public void printReceipt() {
8          System.out.println("Payment of $" + amount + " processed successfully.");
9      }
10 }
11 class CreditCardPayment extends Payment {
12     private String cardNumber;
13     public CreditCardPayment(double amount, String cardNumber) {
14         super(amount);
15         this.cardNumber = cardNumber;
16     }
17     @Override
18     void processPayment() {
19         System.out.println("Processing Credit Card payment for card ending in "
20             + cardNumber.substring(cardNumber.length() - 4));
21     }
22 }
23 class UPIPayment extends Payment {
24     private String upiId;
25     public UPIPayment(double amount, String upiId) {
26         super(amount);
27         this.upiId = upiId;
28     }
29     @Override
30     void processPayment() {
31         System.out.println("Processing UPI payment via ID: " + upiId);
32     }
33 }

```

```

34 public class MainPayment {
35     Run main | Debug main
36     public static void main(String[] args) {
37         Payment cc = new CreditCardPayment(150.75, "4111222233334444");
38         cc.processPayment();
39         cc.printReceipt();
40
41         System.out.println();
42
43         Payment upi = new UPIPayment(49.99, "user@upi");
44         upi.processPayment();
45         upi.printReceipt();
46     }

```

Output:

```
Listening on 51245
User program running
Processing Credit Card payment for card ending in 4444
Payment of $150.75 processed successfully.

Processing UPI payment via ID: user@upi
Payment of $49.99 processed successfully.
User program finished
```

2. Create an abstract **FoodOrder** class with an abstract method `calculateBill()`. Implement two subclasses, **DineInOrder** and **TakeAwayOrder**, to calculate and display the total bill based on the order type.

Code:

```
1  abstract class FoodOrder {
2      protected double itemTotal;
3      public FoodOrder(double itemTotal) {
4          this.itemTotal = itemTotal;
5      }
6      abstract double calculateBill();
7  }
8  class DineInOrder extends FoodOrder {
9      private static final double SERVICE_CHARGE = 0.10;
10     public DineInOrder(double itemTotal) {
11         super(itemTotal);
12     }
13     @Override
14     double calculateBill() {
15         return itemTotal + (itemTotal * SERVICE_CHARGE);
16     }
17 }
18 class TakeAwayOrder extends FoodOrder {
19     private static final double PACKAGING_FEE = 5.0;
20     public TakeAwayOrder(double itemTotal) {
21         super(itemTotal);
22     }
23     @Override
24     double calculateBill() {
25         return itemTotal + PACKAGING_FEE;
26     }
27 }
28 public class MainFoodOrder {
29     Run main | Debug main
30     public static void main(String[] args) {
31         FoodOrder dineIn = new DineInOrder(50.0);
32         System.out.println("Dine-In Total Bill: $" + dineIn.calculateBill());
33         FoodOrder takeAway = new TakeAwayOrder(50.0);
34         System.out.println("Take-Away Total Bill: $" + takeAway.calculateBill());
35     }
36 }
```

Output:

```
Listening on 57741
User program running
Dine-In Total Bill: $55.0
Take-Away Total Bill: $55.0
User program finished
```